



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

# Week 5 Task Report

---

Malware Analysis · Interactive Sandboxing

Detection Engineering · Incident Response

Prepared by Malaika Azhar · Week 5 (Days 29–35)

# Table of Contents

- 01** Executive Summary.....3
- 02** Week 5 Objectives.....4
- 03** Days 29–30 — Sample Acquisition & Environment Lockdown.....5
- 04** Day 31 — Static Malware Analysis.....9
- 05** Days 32–33 — Interactive Dynamic Analysis (ANY.RUN).....13
- 06** Day 34 — Detection Engineering (Suricata).....20
- 07** Day 34–35 — Wazuh Rules & Incident Response Plan (Status).....23
- 08** Technical SOC Summary.....25
- 09** Executive Summary (Leadership).....26
- 10** Key Learnings & Reflection.....27
- 11** Closing Statement & Sign-Off.....28
- A** Appendix — Deliverables & Screenshot Index.....29

# Executive Summary

## Overview of Week 5 Deliverables

This report documents Week 5 of the Blue Team Internship, covering Days 29 through 35. Where Weeks 1–4 built a defensive operations environment — SIEM, network intrusion detection, a perimeter firewall, threat intelligence enrichment, and vulnerability visibility — Week 5 turned to the threat itself: two real malware samples were acquired, statically and dynamically analysed inside an airgapped Kali VM, and the resulting indicators were converted into working Suricata detection logic.

Two samples were deliberately chosen to cover different attack patterns: njRAT (a Remote Access Trojan using domain-based Dynamic DNS command-and-control) and a Zeus/ZeroAccess sample (a banking-trojan-family dropper using hardcoded-IP peer-to-peer command-and-control disguised as DNS traffic). Both were downloaded from theZoo repository, hashed before any handling, and analysed only after the lab environment was verified airgapped at the hypervisor level.

Static analysis using file, exiftool, binwalk, and strings (both ASCII and Unicode passes) established an initial indicator baseline and cross-verified both samples against VirusTotal. Interactive dynamic analysis in ANY.RUN — run from inside the Kali VM's browser per isolation requirements — recovered the full behavioural picture static analysis could not reach: process injection chains, Startup-folder persistence, a live firewall bypass command, and two structurally different command-and-control mechanisms.

Day 34 converted these findings into detection logic: a custom Suricata rule for njRAT's DynDNS callback was written, loaded, and verified working both alongside the full ET ruleset and in isolation — satisfying the task's independent-verification requirement. A hardware failure on the analysis machine (recurring motherboard-level instability under VM load) interrupted the remainder of Day 34 and all of Day 35; this is documented transparently below rather than fabricated, consistent with this internship's reporting standard.

| Task Block                                            | Status      | Notes                                                               |
|-------------------------------------------------------|-------------|---------------------------------------------------------------------|
| <b>Days 29–30 — Sample Acquisition &amp; Lockdown</b> | Complete    | Two theZoo samples acquired, hashed, airgapped, permission-locked   |
| <b>Day 31 — Static Malware Analysis</b>               | Complete    | file/exiftool/binwalk/strings run on both samples, VT cross-checked |
| <b>Days 32–33 — Dynamic Analysis (ANY.RUN)</b>        | Complete    | Both samples detonated interactively, full IOC/IOA set extracted    |
| <b>Day 34 Part 1 — Suricata Detection</b>             | Partial     | njRAT custom rule written & verified; Zeus rule drafted, untested   |
| <b>Day 34 Part 2 — Wazuh Custom Rules</b>             | Not Started | Blocked by hardware failure on analysis machine                     |
| <b>Day 35 — Incident Response Plan</b>                | Not Started | Depends on Wazuh rules above; all supporting evidence is ready      |

As with previous weeks, this report documents the troubleshooting encountered along the way — including a rule saved as a disabled comment, a SID collision with an earlier week's custom rule, and a hardware fault that halted work mid-week — rather than presenting the week as friction-free.



# Week 5 Objectives

Days 29–35

Week 4 closed with a lab capable of enforcing a managed network perimeter, enriching alerts with real threat intelligence, tracking CVE exposure, and reporting to two different audiences. What remained missing, identified explicitly at the start of the Week 5 training, was any first-hand experience with the threat itself — a SOC analyst who only ever works from alerts a SIEM produces is structurally limited compared to one who can extract indicators from a live sample and translate them directly into detection rules. Week 5 was scoped to close that gap.

## Learning Outcomes Targeted

- Safely acquire and evaluate real malware samples, and explain the connect → download → verify → airgap → analyse sequence and why reversing any step introduces risk.
- Apply static analysis tooling (file, exiftool, binwalk, strings) and distinguish what each tool reveals, including recognising when a near-empty result itself signals a packed/obfuscated sample.
- Use an interactive cloud sandbox to recover runtime behaviour static analysis cannot reach — process injection, persistence, firewall manipulation, and command-and-control mechanics.
- Translate observed IOCs and IOAs into working Suricata signatures, and prove those signatures fire independently of the default ruleset via isolated replay.
- Distinguish what Suricata (network layer) can and cannot detect versus what requires Wazuh (host layer), and map both to specific MITRE ATT&CK techniques.

### SCOPE NOTE

*This report follows the four-block structure specified in the Week 5 training: Days 29–30 (sample acquisition and environment lockdown), Day 31 (static malware analysis), Days 32–33 (interactive dynamic analysis), and Day 34–35 (detection engineering and incident response). Each section documents what was done, what was observed, what was concluded, and any troubleshooting undertaken along the way — including work that could not be completed this reporting period.*

# Days 29–30 — Sample Acquisition & Environment Lockdown

*Concepts: sample selection · hash-first identification · hypervisor-level airgap*

## Objective

Select two malware samples representing different categories, acquire them safely from theZoo repository without cloning the full repo, compute hashes before any further handling, then lock the lab environment down to a verified airgap before any analysis begins.

## Sample Choice Justification

Two samples were selected to cover different C2 architectures and give Days 34–35 two distinct detection engineering problems:

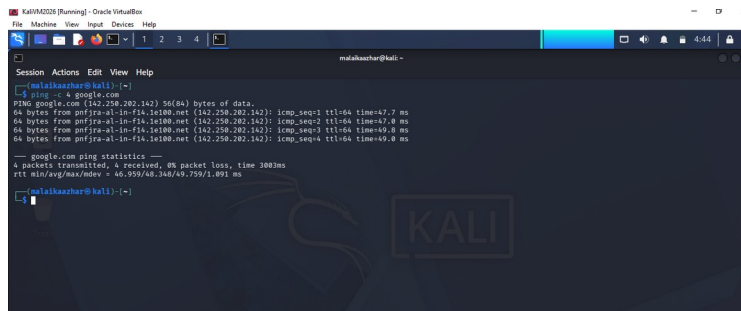
**njRAT v0.6.4** — a Remote Access Trojan chosen for its domain-based Dynamic DNS command-and-control and its prevalence in real-world credential-theft campaigns.

**Zeus Banking Trojan (26 Nov 2013 build)** — chosen to contrast with njRAT: this sample's actual detonation showed it belongs to the ZeroAccess/Sirefef family (confirmed on VirusTotal), using a hardcoded IP and encrypted peer-to-peer traffic disguised as DNS, rather than the domain-based mechanism njRAT uses.

Both samples were sourced from theZoo GitHub repository, downloading only the individual ZIP and hash-verification files for each sample path — not a full repository clone, per the task requirement.

## Acquisition Evidence

Kali VM internet connectivity was confirmed before any download attempt:



*Figure 3.1 — ping -c 4 google.com returns successfully (0% packet loss), confirming the Kali VM has an active internet path before acquisition begins.*

A dedicated sample directory was created and locked to owner-only access before any file was placed inside it:

```
rmkdir ~/malware_samples
mkdir ~/malware_samples
chmod 700 ~/malware_samples
ls -ld ~/malware_samples
```

```

(malaikaazhar@kali)-[~]
└─$ mkdir ~/malware_samples
└─$ chmod 700 ~/malware_samples
└─$ ls -ld ~/malware_samples
drwx----- 2 malaikaazhar malaikaazhar 4096 Aug 20 20:49 /home/malaikaazhar/malware_samples
└─$

```

Figure 3.2 — `ls -ld` confirms `drwx-----` (700) on `~/malware_samples`: readable, writable, and executable only by the owner.

Both sample folders were located on theZoo's GitHub repository by browsing directly to each sample's subdirectory:

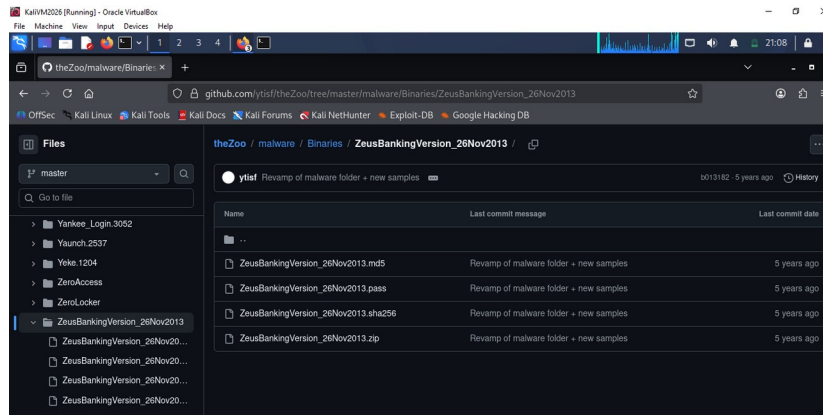


Figure 3.3 — theZoo repository, ZeusBankingVersion\_26Nov2013 folder, showing the four component files (.md5, .pass, .sha256, .zip).

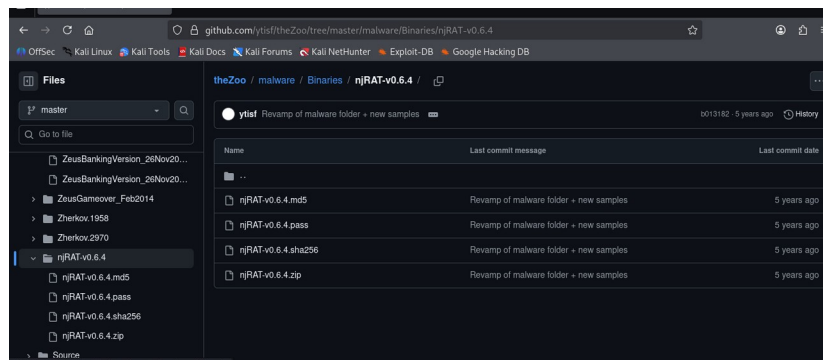


Figure 3.4 — theZoo repository, njRAT-v0.6.4 folder, same four-file structure.

Each file was downloaded individually via `wget` — not `git clone` — directly from the raw GitHub content path:

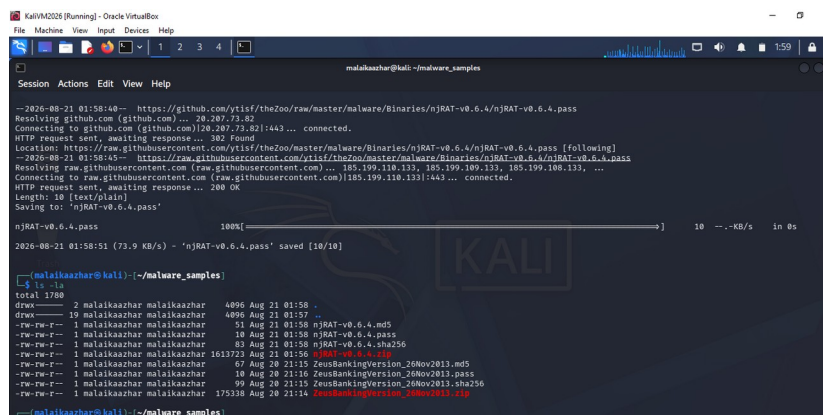


Figure 3.5 — `wget` downloading `njRAT-v0.6.4.pass` individually (100% complete, 10 bytes), with `ls -la` confirming all four component files present for both samples in the working directory.

Both ZIPs were extracted using theZoo's standard archive password, infected:

```
unzip -P infected njRAT-v0.6.4.zip
```

```
(malaikaazhar@kali)-[~/malware_samples]
$ unzip -P infected njRAT-v0.6.4.zip
Archive:  njRAT-v0.6.4.zip
  creating: njRAT-v0.6.4/
  inflating: njRAT-v0.6.4/GeoIP.dat
  inflating: njRAT-v0.6.4/Mono.Cecil.dll
  inflating: njRAT-v0.6.4/NAudio.dll
  inflating: njRAT-v0.6.4/njRAT.exe
   creating: njRAT-v0.6.4/Plugin/
  inflating: njRAT-v0.6.4/Plugin/cam.dll
  inflating: njRAT-v0.6.4/Plugin/ch.dll
  inflating: njRAT-v0.6.4/Plugin/fm.dll
  inflating: njRAT-v0.6.4/Plugin/Mic.dll
  inflating: njRAT-v0.6.4/Plugin/pw.dll
  inflating: njRAT-v0.6.4/Plugin/sc2.dll
  inflating: njRAT-v0.6.4/stub.il
  inflating: njRAT-v0.6.4/Stub.manifest
(malaikaazhar@kali)-[~/malware_samples]
$
```

Figure 3.6 — unzip output confirming successful extraction of the full njRAT-v0.6.4 folder structure, including the Plugin/ subfolder (cam.dll, ch.dll, fm.dll, Mic.dll, pw.dll, sc2.dll).

## Hashes (computed immediately post-extraction)

| Sample                    | MD5                                  | SHA256                                                                               |
|---------------------------|--------------------------------------|--------------------------------------------------------------------------------------|
| Zeus (invoice_...pdf.exe) | ea039a854d20d7734c5add48f1a51c34     | 69e966e730557fde8fd84317cdef1ece00a8b<br>bb3470c0b58f3231e170168af169 (VT-confirmed) |
| njRAT (njRAT.exe)         | 0431311b5f024d6e66b90d59491f256<br>3 | fd624aa205517580e83fad7a4ce4d64863e9<br>5f62b34ac72647b1974a52822199                 |

```
malaikaazhar@kali: ~/malware_samples
Session Actions Edit View Help
(malaikaazhar@kali)-[~/malware_samples]
$ md5sum invoice_2318362983713_82393134210.pdf.exe
ea039a854d20d7734c5add48f1a51c34  invoice_2318362983713_82393134210.pdf.exe
(malaikaazhar@kali)-[~/malware_samples]
$ cat ZeusBankingVersion_26Nov2013.md5
cat ZeusBankingVersion_26Nov2013.sha256
858e2aed6ba9b096679967da540d40c3 ZeusBankingVersion_26Nov2013.zip
4644b5fb10fb84cd47bec4b5a48d5e60165e8ae2130fca5c055633aaad73162 ZeusBankingVersion_26Nov2013.zip
(malaikaazhar@kali)-[~/malware_samples]
$ sha256sum ZeusBankingVersion_26Nov2013.zip
md5sum ZeusBankingVersion_26Nov2013.zip
4644b5fb10fb84cd47bec4b5a48d5e60165e8ae2130fca5c055633aaad73162 ZeusBankingVersion_26Nov2013.zip
858e2aed6ba9b096679967da540d40c3 ZeusBankingVersion_26Nov2013.zip
(malaikaazhar@kali)-[~/malware_samples]
$ sha256sum njRAT-v0.6.4/njRAT.exe
md5sum njRAT-v0.6.4/njRAT.exe
sha256sum njRAT-v0.6.4.zip
md5sum njRAT-v0.6.4.zip
fd624aa205517580e83fad7a4ce4d64863e95f62b34ac72647b1974a52822199 njRAT-v0.6.4/njRAT.exe
0431311b5f024d6e66b90d59491f2563 njRAT-v0.6.4/njRAT.exe
5ff121c57e4a2f27f5e4985660c9666a44b39ef2549b29b3a4d6a1e06e6e3f65 njRAT-v0.6.4.zip
3ccce9d87ce9ea751abea094d1639d0a njRAT-v0.6.4.zip
(malaikaazhar@kali)-[~/malware_samples]
$ cat njRAT-v0.6.4.sha256
cat njRAT-v0.6.4.md5
5ff121c57e4a2f27f5e4985660c9666a44b39ef2549b29b3a4d6a1e06e6e3f65 njRAT-v0.6.4.zip
3ccce9d87ce9ea751abea094d1639d0a njRAT-v0.6.4.zip
(malaikaazhar@kali)-[~/malware_samples]
$
```

Figure 3.7 — md5sum/sha256sum output for both samples and both ZIP archives, cross-checked directly against theZoo's published .md5/.sha256 files via cat — values match exactly, confirming download integrity.

## Airgap and Permission Lockdown (Day 30)

With both samples downloaded, verified, and hashed, the NAT adapter was disabled at the hypervisor level (VirtualBox settings), not from inside the guest OS — because disabling a NIC inside the VM still leaves the hypervisor's virtual attachment active.

```
(malaikaazhar@kali)-[~/malware_samples]
$ ping -c 4 google.com
ping: google.com: Temporary failure in name resolution

(malaikaazhar@kali)-[~/malware_samples]
$
```

Figure 3.8 — `ping -c 4 google.com` returns "Temporary failure in name resolution" — a failed test, which is the correct airgap confirmation. A passing ping here would have proven nothing; only a failure confirms isolation.

Final restrictive permissions were then applied to both extracted sample executables:

```
chmod 400 ~/malware_samples/invoice_2318362983713_823931342io.pdf.exe
chmod 400 ~/malware_samples/njRAT-v0.6.4/njRAT.exe
```

```
(malaikaazhar@kali)-[~/malware_samples]
$ chmod 400 ~/malware_samples/invoice_2318362983713_823931342io.pdf.exe
$ chmod 400 ~/malware_samples/njRAT-v0.6.4/njRAT.exe
$ ls -la ~/malware_samples/
ls -la ~/malware_samples/njRAT-v0.6.4/
total 2032
drwx----- 3 malaikaazhar malaikaazhar 4096 Aug 21 02:01 .
drwx----- 19 malaikaazhar malaikaazhar 4096 Aug 21 01:57 ..
-r----- 1 malaikaazhar malaikaazhar 252928 Nov 26 2013 invoice_2318362983713_823931342io.pdf.exe
drwxrwxr-x 3 malaikaazhar malaikaazhar 4096 Sep 27 2013 njRAT-v0.6.4
-rw-rw-r-- 1 malaikaazhar malaikaazhar 51 Aug 21 01:58 njRAT-v0.6.4.md5
-rw-rw-r-- 1 malaikaazhar malaikaazhar 10 Aug 21 01:58 njRAT-v0.6.4.pass
-rw-rw-r-- 1 malaikaazhar malaikaazhar 83 Aug 21 01:58 njRAT-v0.6.4.sha256
-rw-rw-r-- 1 malaikaazhar malaikaazhar 1613723 Aug 21 01:56 njRAT-v0.6.4.zip
-rw-rw-r-- 1 malaikaazhar malaikaazhar 67 Aug 20 21:15 ZeusBankingVersion_26Nov2013.md5
-rw-rw-r-- 1 malaikaazhar malaikaazhar 10 Aug 20 21:16 ZeusBankingVersion_26Nov2013.pass
-rw-rw-r-- 1 malaikaazhar malaikaazhar 99 Aug 20 21:15 ZeusBankingVersion_26Nov2013.sha256
-rw-rw-r-- 1 malaikaazhar malaikaazhar 175338 Aug 20 21:14 ZeusBankingVersion_26Nov2013.zip
total 3244
drwxrwxr-x 3 malaikaazhar malaikaazhar 4096 Sep 27 2013 .
drwx----- 3 malaikaazhar malaikaazhar 4096 Aug 21 02:01 ..
-rw-rw-r-- 1 malaikaazhar malaikaazhar 1296658 Jul 6 2013 GeoIP.dat
-rw-rw-r-- 1 malaikaazhar malaikaazhar 312320 Mar 11 2012 Mono.Cecil.dll
-rw-rw-r-- 1 malaikaazhar malaikaazhar 391168 Dec 18 2011 NAudio.dll
-r----- 1 malaikaazhar malaikaazhar 982016 Sep 27 2013 njRAT.exe
drwxrwxr-x 2 malaikaazhar malaikaazhar 4096 Sep 27 2013 Plugin
-rw-rw-r-- 1 malaikaazhar malaikaazhar 315222 Sep 24 2013 stub.il
-rw-rw-r-- 1 malaikaazhar malaikaazhar 487 Aug 17 2013 Stub.manifest
(malaikaazhar@kali)-[~/malware_samples]
$
```

Figure 3.9 — `ls -la` confirming both sample executables show `-r-----` (400: read-only, non-executable, owner-only), while the top-level `malware_samples` directory remains locked at `drwx-----` (700).

A baseline network capture was taken on the Host-Only interface before any analytical activity began, to establish what "normal" traffic looks like in the airgapped lab before the samples are touched:

```
sudo tcpdump -i eth0 -w baseline_capture.pcap
```

```
valid_lft forever preferred_lft forever
(malaikaazhar@kali)-[~/malware_samples]
$ sudo tcpdump -i eth0 -w baseline_capture.pcap
[sudo] password for malaikaazhar:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

^C61 packets captured
61 packets received by filter
0 packets dropped by kernel
(malaikaazhar@kali)-[~/malware_samples]
$ ls -la baseline_capture.pcap
-rw-r--r-- 1 tcpdump tcpdump 4410 Aug 21 02:17 baseline_capture.pcap
(malaikaazhar@kali)-[~/malware_samples]
$
```

Figure 3.10 — `tcpdump` capturing 61 packets on `eth0` over several minutes of idle observation, saved as `baseline_capture.pcap` (4,410 bytes).

### Why are hashes computed before any other handling, and used as the identifier instead of filenames?

A filename is arbitrary and trivially changed by anyone, including the malware author disguising the file. A cryptographic hash is a fingerprint of the actual binary content — it cannot be faked by renaming, and it is the value every other tool (VirusTotal, threat feeds, IR documentation) will actually recognise. Computing it immediately after extraction, before any tool touches the file, guarantees the hash on record corresponds to the exact bytes that were analysed.

### Why must the airgap be verified at the hypervisor level rather than confirmed from inside the guest OS?

Bringing a network interface down inside the VM only tells the guest OS to stop using it — the hypervisor's virtual switch attachment for that adapter can still be active underneath. The only way to guarantee the sample truly cannot reach the internet, even if it tries to reset or re-enable its own network stack, is to remove the adapter's connection at the VirtualBox settings level, outside the guest's control entirely.

### DELIVERABLES SUBMITTED

Kali connectivity confirmation · 700-permission directory listing · theZoo repository browsing screenshots (both samples) · individual wget downloads (no git clone) · extraction confirmation · hash table cross-checked against theZoo's published values · failed-ping airgap confirmation · 400-permission sample listing · baseline PCAP capture

# Day 31 — Static Malware Analysis

Concepts: file-type identification · metadata inspection · embedded payload detection · ASCII vs. Unicode strings

## Objective

Examine both samples without executing them — cataloguing what each file contains, what it suggests, and which questions dynamic analysis will need to answer — using file, exiftool, binwalk, and strings (both ASCII and 16-bit little-endian Unicode passes).

## file — File-Type Identification

```
(malaikaazhar@kali)~[~/malware_samples]
$ file invoice_2318362983713_823931342io.pdf.exe
file: invoice_2318362983713_823931342io.pdf.exe: PE32 executable for MS Windows 5.01 (GUI), Intel i386, 6 sections
file: njRAT-v0.6.4/njRAT.exe: PE32 executable for MS Windows 4.00 (GUI), Intel i386 Mono/.Net assembly, 4 sections
(malaikaazhar@kali)~[~/malware_samples]
$
(malaikaazhar@kali)~[~/malware_samples]
```

Figure 4.1 — file output for both samples: the Zeus sample is a 32-bit Windows PE despite its .pdf.exe disguise; njRAT is a .NET/Mono assembly, consistent with njRAT's known C# codebase.

Interpretation: Both are 32-bit Windows PE executables. The Zeus sample's double extension (.pdf.exe) is a classic masquerading trick — Windows hides known file extensions by default, so to an unwary user the file appears to be a harmless PDF.

## exiftool — Metadata Inspection

```
Session Actions Edit View Help
L-$
(malaikaazhar@kali)~[~/malware_samples]
$ exiftool invoice_2318362983713_823931342io.pdf.exe
exiftool: njRAT-v0.6.4/njRAT.exe
ExifTool Version Number      : 13.55
File Name                    : invoice_2318362983713_823931342io.pdf.exe
Directory                    : .
File Size                     : 253 kB
File Modification Date/Time   : 2013:11:26 14:01:00+05:00
File Access Date/Time        : 2026:08:21 02:20:45+05:00
File Inode Change Date/Time   : 2026:08:21 02:11:28+05:00
File Permissions              : -r-----
File Type                     : Win32 EXE
File Type Extension           : exe
MIME Type                     : application/octet-stream
Machine Type                  : Intel 386 or later, and compatibles
Time Stamp                    : 2013:11:25 15:32:03+05:00
Image File Characteristics    : Executable, 32-bit
PE Type                       : PE32
Linker Version                : 10.0
Code Size                    : 144384
Initialized Data Size         : 107520
Uninitialized Data Size       : 0
Entry Point                   : 0*a3b6
OS Version                    : 5.1
Image Version                 : 0.0
Subsystem Version             : 5.1
Subsystem                     : Windows GUI
ExifTool Version Number      : 13.55
File Name                    : njRAT.exe
Directory                    : njRAT-v0.6.4
File Size                     : 982 kB
File Modification Date/Time   : 2013:09:27 11:00:20+05:00
```

Figure 4.2 — exiftool run against both samples: Zeus compile timestamp of 2013-11-25 with no code-signing certificate; njRAT's internal original filename confirmed as njRAT.exe (982 KB), separate from the outer wrapper.

Interpretation: The Zeus sample is unsigned and its metadata is consistent with an old, unmaintained banking-trojan build. njRAT's internal filename differing from any wrapper name it might be delivered under is itself a useful static indicator — the real payload identity survives inside the metadata even if the outer file is renamed.

## binwalk — Embedded Payload Detection

```
(malaikaazhar@kali) - [~/malware_samples]
$ binwalk -e njRAT-v0.6.4/njRAT.exe
ls -la _njRAT.exe.extracted/

DECIMAL      HEXADECIMAL    DESCRIPTION
-----
211005      0x3383D        Zlib compressed data, default compression
779036      0xBE31C        Zlib compressed data, compressed

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented
ls: cannot access '_njRAT.exe.extracted/': No such file or directory

(malaikaazhar@kali) - [~/malware_samples]
$
```

Figure 4.3 — binwalk against njRAT.exe identifies two embedded Zlib-compressed blobs at offsets 0x3383D and 0xBE31C; extraction failed with "no utility was found or it's unimplemented."

Interpretation: The two compressed blobs are consistent with njRAT's known plugin architecture — the extracted ZIP's Plugin/ folder (cam.dll, ch.dll, fm.dll, Mic.dll, pw.dll, sc2.dll) confirms njRAT bundles its capability modules this way. Extraction failing is a tooling/environment limitation on this Kali installation, not evidence the payload is absent — this is noted honestly as a gap rather than glossed over: the presence of compressed content is confirmed, its contents are not.

## strings — ASCII vs. Unicode (-el) Comparison

```
(malaikaazhar@kali) - [~/malware_samples]
$ cat strings_unicode_zeus.txt
! "$%&'()*+,-./0123456789:;<

(malaikaazhar@kali) - [~/malware_samples]
$
```

Figure 4.4 — cat of the Zeus Unicode strings output: only 30 bytes total, consisting solely of punctuation characters — no domains, paths, or readable content of any kind.

| Sample | ASCII strings size | Unicode (-el) strings size    |
|--------|--------------------|-------------------------------|
| Zeus   | 147,205 bytes      | 30 bytes (punctuation only)   |
| njRAT  | 63,436 bytes       | 3,593 bytes (genuine content) |

Interpretation: The near-total absence of Unicode content in the Zeus sample is itself a finding, not a null result. It strongly suggests the sample is packed — its real configuration data (C2 addresses, strings) is compressed or encrypted and only becomes readable in memory at runtime. This directly motivated the emphasis on capturing runtime behaviour closely during Days 32–33.

## VirusTotal Cross-Verification

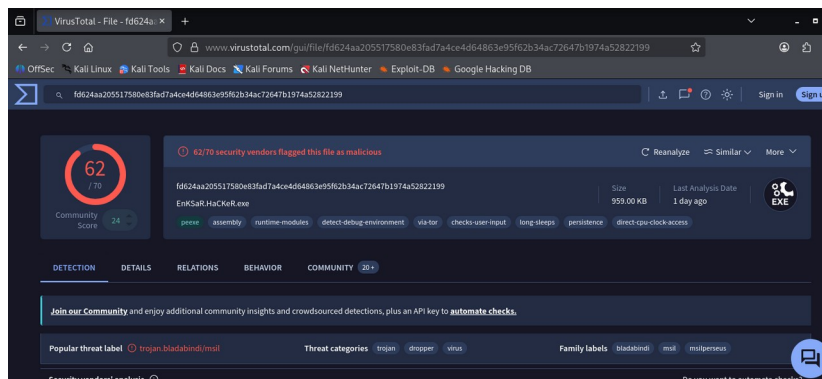


Figure 4.5 — VirusTotal: njRAT sample flagged 62/70, popular threat label trojan.bladabindi/msil, family tags bladabindi / msil / msilperseus.

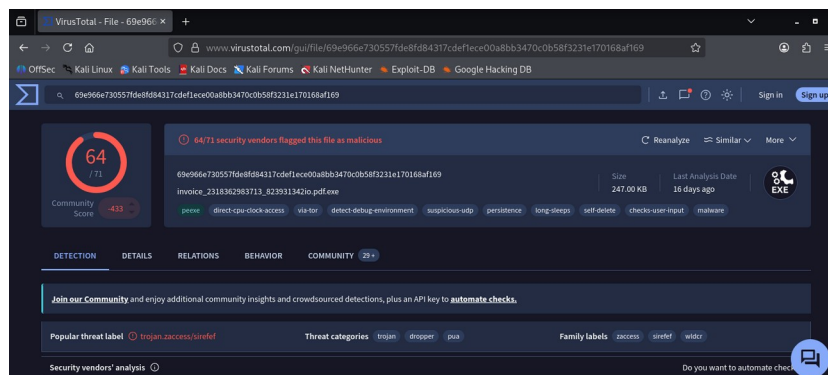


Figure 4.6 — VirusTotal: Zeus/invoice sample flagged 64/71, popular threat label trojan.zaccess/sirefef, family tags zaccess / sirefef / wldcr.

Interpretation: VT confirms both samples are widely known, community-flagged threats — but notably, the Zeus-named sample's dominant classification across vendors is actually ZeroAccess/Sirefef, not classic Zeus banking-injection malware. The theZoo repository's folder name reflects its own labelling convention; the payload's true family, confirmed independently by both VT consensus and the Day 32–33 dynamic behaviour, is ZeroAccess.

## Initial IOC List (Static-Analysis Stage)

| Indicator Type    | Value                            | Source            | Significance                   |
|-------------------|----------------------------------|-------------------|--------------------------------|
| SHA256            | fd624aa2...822199                | njRAT hash        | Unique sample identifier       |
| SHA256            | 69e966e7...168af169              | Zeus hash (VT)    | Unique sample identifier       |
| File anomaly      | .pdf.exe double extension        | file, Zeus        | Social-engineering masquerade  |
| Embedded payload  | 2x Zlib blobs, 0x3383D / 0xBE31C | binwalk, njRAT    | Packed plugin DLLs             |
| Packing indicator | Near-empty Unicode strings       | strings -el, Zeus | Sample likely packed/encrypted |

### What does the -el flag do in the strings command, and why does the Unicode scan produce different output from the ASCII scan?

The default strings scan looks only for sequences of 7-bit ASCII printable characters. The -el flag instead scans for 16-bit little-endian Unicode (UTF-16LE) sequences — the encoding Windows programs commonly use internally for text, including many API calls and configuration strings. In this analysis the difference was stark: njRAT's Unicode pass revealed genuine content beyond its ASCII pass, while Zeus's Unicode pass revealed almost nothing at all — demonstrating that a single scan mode can easily miss an entire layer of a sample's stored content depending on how that content was encoded.

### What is a packed executable, why are packers used, and how does binwalk detect embedded payloads?

A packed executable stores its real code and data in a compressed or encrypted form inside the outer file, decompressing itself into memory only at runtime — this defeats static string and signature scanning, since

the interesting content simply isn't present in readable form until the program actually executes. binwalk detects this by scanning the binary for known file-format signatures (magic bytes) at arbitrary offsets, rather than assuming content only exists at the structure a normal PE parser expects — this is exactly how it located the two Zlib-compressed sections buried inside njRAT.exe.

#### **DELIVERABLES SUBMITTED**

file output for both samples · exiftool metadata for both samples · binwalk output showing embedded Zlib payloads · ASCII vs. Unicode strings comparison · VirusTotal screenshots for both samples · static-stage IOC table

# Days 32–33 — Interactive Dynamic Analysis (ANY.RUN)

Concepts: interactive sandboxing · process injection · persistence mechanisms · C2 architecture comparison

## Objective

Execute both samples in ANY.RUN, an interactive cloud sandbox, from inside the Kali VM's browser (not the host machine, to avoid host-side AV interference), with Fake Net enabled to capture C2 traffic even though the real infrastructure behind decade-old samples is long offline. Both sessions ran on Windows 10 (64-bit), Public visibility, with manual verification that detonation actually occurred.

## Sample 1: njRAT.exe

**ANY.RUN Task:** [app.any.run/tasks/a129669d-1d0b-42fb-b50c-cd1c8047dc34](https://app.any.run/tasks/a129669d-1d0b-42fb-b50c-cd1c8047dc34) | **Verdict:** Malicious activity — **Danger score 100/100** | **Runtime:** 71s

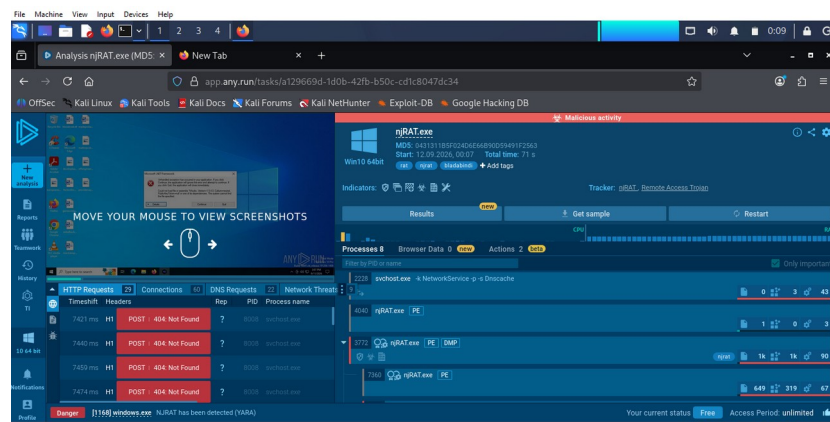


Figure 5.1 — njRAT session overview: verdict banner "Malicious activity," tracker tags njRAT / Remote Access Trojan, first process-tree segment.

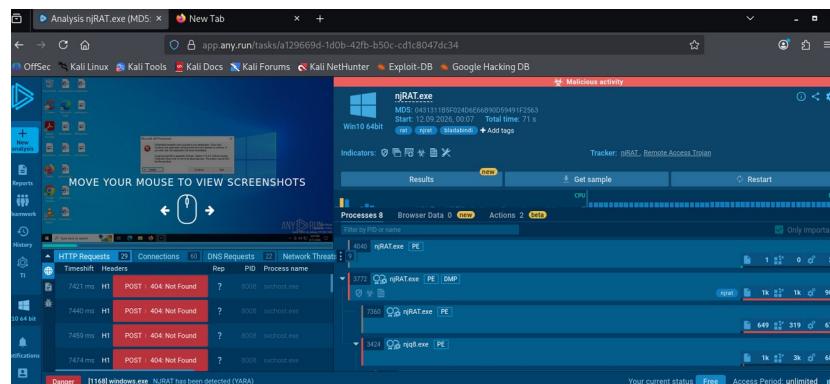


Figure 5.2 — Process tree continued, revealing njRAT self-copying under a new process name, njq8.exe.

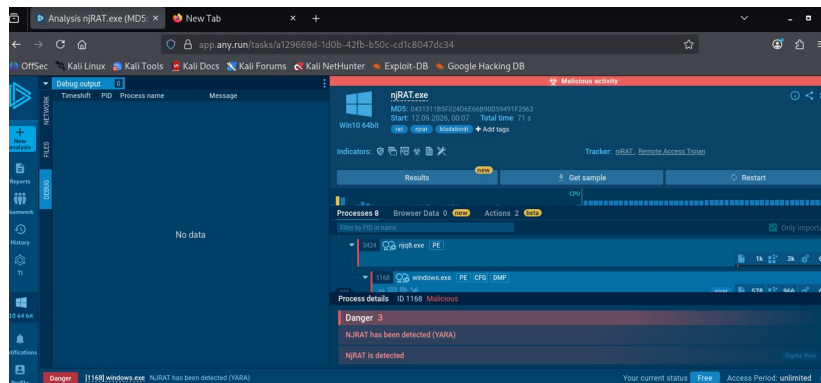


Figure 5.3 — Full process chain resolved: windows.exe (PID 1168) spawns netsh.exe with a firewall allow-rule command, plus conhost.exe.

Process chain observed: njRAT.exe → njRAT.exe → njRAT.exe → njq8.exe → windows.exe → { netsh.exe, conhost.exe }. Key finding: njRAT progressively renames its own copies (njRAT.exe → njq8.exe → windows.exe) — a basic evasion technique against filename-based detection.

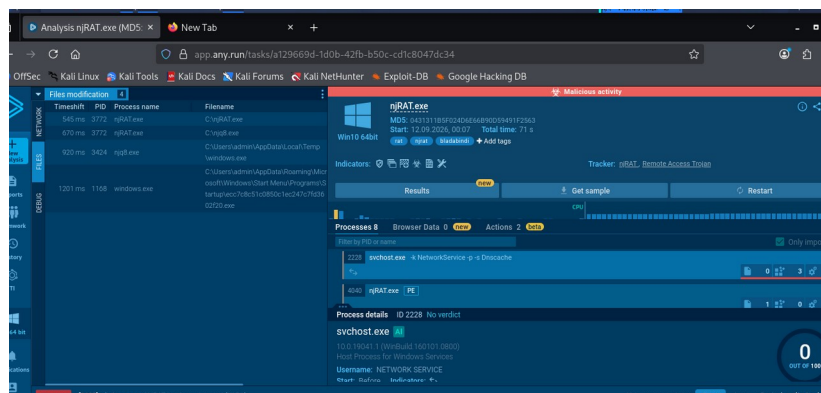


Figure 5.4 — File-system activity: final drop lands in the Startup folder under a randomized hex filename, ecc7c8c51c0850c1ec247c7fd3602f20.exe.

Key finding — persistence: the Startup-folder drop with a randomized filename ensures the malware relaunches on every login without needing a registry Run key at all.

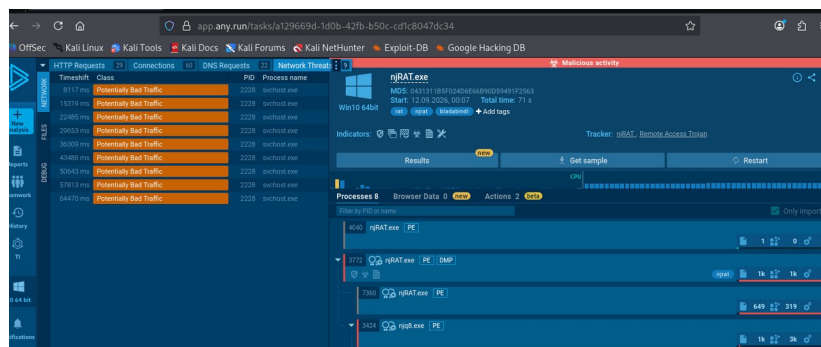


Figure 5.5 — Network Threats tab: 10 flagged events, all "Potentially Bad Traffic," tied to process svchost.exe / PID 2228 (the Windows DNS cache service acting as the resolver for njRAT's query).

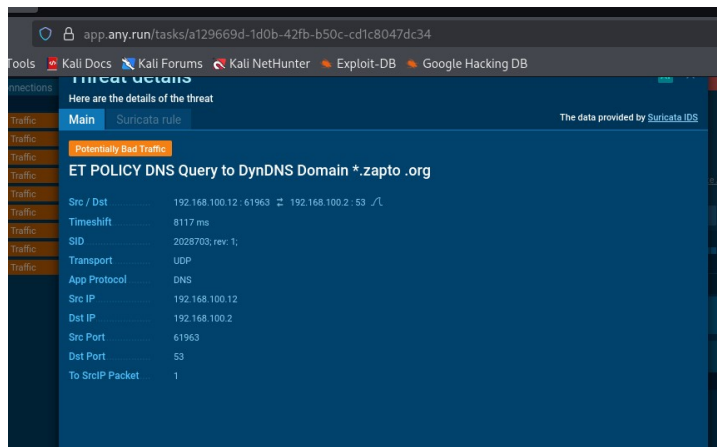


Figure 5.6 — Threat detail expansion: "ET POLICY DNS Query to DynDNS Domain \*.zaproto.org," SID 2028703, UDP, source 192.168.100.12:61963 → destination 192.168.100.2:53.

Key finding — C2 via Dynamic DNS: njRAT resolves its command-and-control address through zapto.org, a free Dynamic DNS provider. This lets the attacker's real IP change freely at any time without needing to update the malware binary or any hardcoded address.

#### Command line captured (Indicator of Attack):

```
netsh.exe firewall add allowedprogram "C:\Users\admin\AppData\Local\Temp\windows.exe" "windows.exe" ENABLE
```

Significance: the malware explicitly opens a hole in Windows Firewall for itself, guaranteeing its outbound traffic to zapto.org is not blocked — a behavioural indicator that no static file signature could ever surface.

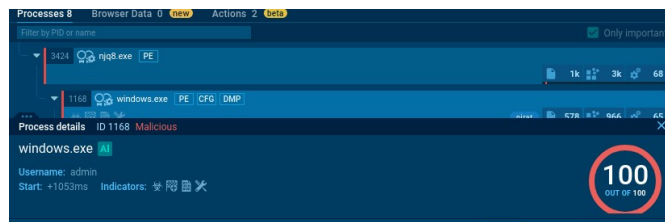


Figure 5.7 — Process details for windows.exe: Danger score 100/100.

MITRE ATT&CK techniques observed for this process (via ANY.RUN's process indicators — the free tier does not expose raw registry key/value pairs, so persistence is documented at the technique level instead):

| Technique | Name                               | Evidence                                                                     |
|-----------|------------------------------------|------------------------------------------------------------------------------|
| T1547.001 | Registry Run Keys / Startup Folder | Autorun value changed; file created in Startup directory                     |
| T1562.004 | Disable or Modify System Firewall  | netsh.exe used to add a firewall allow-rule                                  |
| T1012     | Query Registry                     | Reads machine GUID from the registry                                         |
| T1082     | System Information Discovery       | Reads computer name, checks supported languages — environment fingerprinting |

Artifacts collected: full PCAP (a129669d-1d0b-42fb-b50c-cd1c8047dc34.pcap), IOCs export (iocs.json), public share link (above).

## Sample 2: invoice\_...\_io.pdf.exe (Zeus / ZeroAccess)

**ANY.RUN Task:** app.any.run/tasks/19c0da7a-8389-47d2-854b-39208744e930 | **Verdict:** Malicious activity — **Danger score 100/100** | **Runtime:** 248s

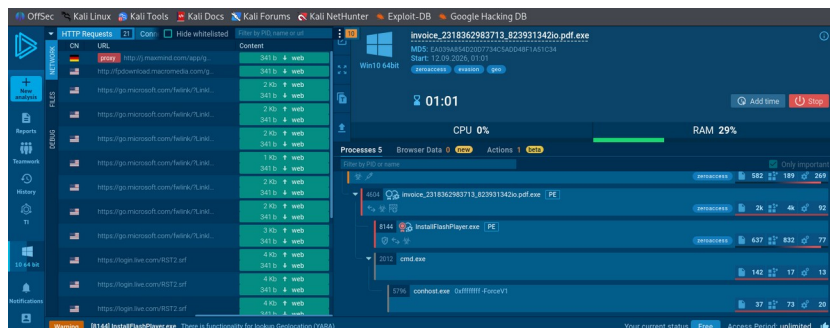


Figure 5.8 — Zeus session overview: tags zeroaccess / evasion / geo, process chain invoice\_...pdf.exe → InstallFlashPlayer.exe → cmd.exe → conhost.exe.

Key finding — fake installer masquerade: the sample drops and runs a file named InstallFlashPlayer.exe, forging Adobe's own version metadata ("Adobe® Flash® Player Installer/Uninstaller 11.0 r1") to avoid raising suspicion during execution.

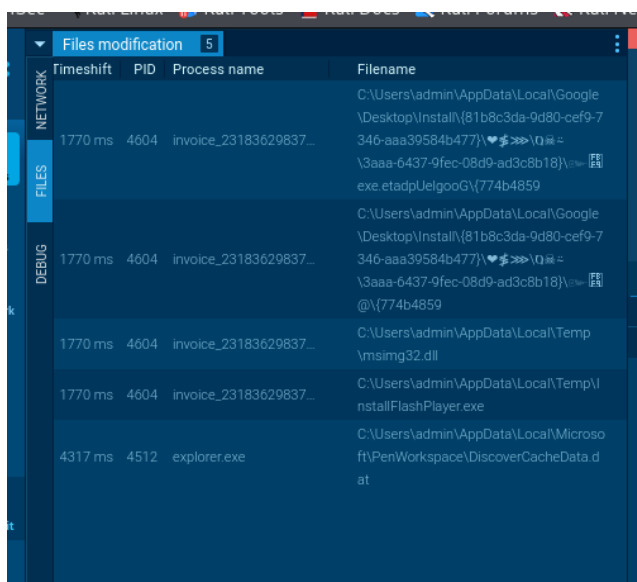


Figure 5.9 — File-system activity: obfuscated GUID-style folder under AppData\Local\Google\Desktop\Install\, plus msimg32.dll and InstallFlashPlayer.exe dropped into Temp.

Key finding — DLL masquerading: msimg32.dll is the name of a legitimate Windows system DLL. Dropping a malicious file under this exact name is consistent with DLL search-order hijacking — positioning a malicious file to be loaded ahead of the genuine system library.

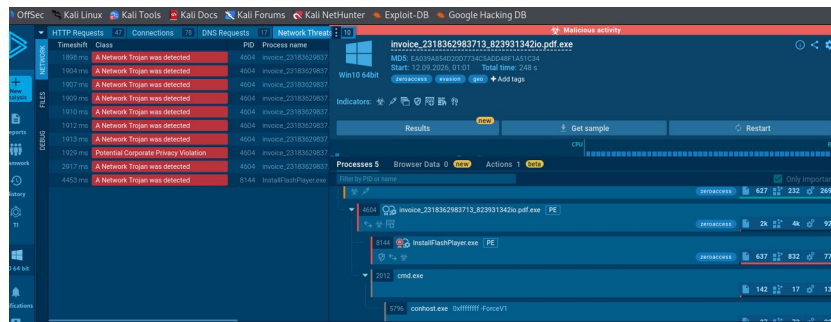


Figure 5.10 — Network Threats tab: 10 "A Network Trojan was detected" hits plus one "Potential Corporate Privacy Violation" flag, consistent with credential/data-exfiltration-oriented banking-trojan-family behaviour.

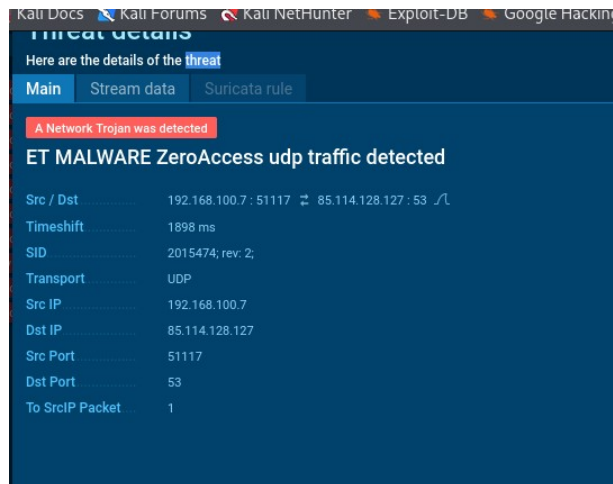


Figure 5.11 — Threat detail expansion: "ET MALWARE ZeroAccess udp traffic detected," SID 2015474 rev 2, source 192.168.100.7:51117 → destination 85.114.128.127:53 (UDP).

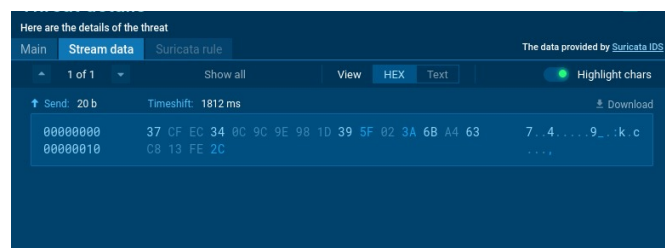


Figure 5.12 — Raw stream payload: 20 bytes, hex 37 CF EC 34 0C 9C 9E 98 1D 39 5F 02 3A 6B A4 63 C8 13 FE 2C — encrypted/obfuscated binary content, not real DNS.

Key finding — C2 via hardcoded IP disguised as DNS: unlike njRAT's domain-based approach, this sample uses a hardcoded IP address (85.114.128.127) and sends encrypted peer-to-peer protocol data over UDP port 53 — deliberately using the DNS port to blend in with normal network noise, without the traffic actually being DNS underneath. This is the ZeroAccess botnet's known P2P C2 architecture, and it changes the shape of the Suricata rule this sample needs relative to njRAT's (IP+port match rather than a domain-content match).



Figure 5.13 — Process details for InstallFlashPlayer.exe: Danger score 100/100.

Additional finding — anti-analysis/geolocation check: a YARA warning flagged "functionality for lookup Geolocation," corroborated by an HTTP request to cj.maxmind.com (a real IP-geolocation service) visible in the HTTP

Requests tab — the sample checks its own apparent location before proceeding, a common technique to avoid detonating inside known sandbox/research IP ranges.

Artifacts collected: full PCAP (task 19c0da7a-...), IOCs export, public share link (above).

## Consolidated IOC / IOA Table (Days 31–33)

| #  | Type               | Value                                 | Sample | Significance                       |
|----|--------------------|---------------------------------------|--------|------------------------------------|
| 1  | SHA256             | fd624aa2...822199                     | njRAT  | Sample identifier                  |
| 2  | SHA256             | 69e966e7...168af169                   | Zeus   | Sample identifier                  |
| 3  | Domain (DynDNS)    | *.zapto.org                           | njRAT  | C2 via free dynamic-DNS provider   |
| 4  | Dest. IP:Port      | 85.114.128.127:53/UDP                 | Zeus   | Hardcoded P2P C2, disguised as DNS |
| 5  | File path          | ...\Startup\ecc7c8c5...02f20.exe      | njRAT  | Persistence, randomized filename   |
| 6  | File path          | ...\Temp\msimg32.dll                  | Zeus   | DLL search-order hijack masquerade |
| 7  | Process rename     | njq8.exe, windows.exe                 | njRAT  | Self-renaming evasion              |
| 8  | Process masquerade | InstallFlashPlayer.exe                | Zeus   | Fake-installer masquerade          |
| 9  | Command line       | netsh firewall add allowedprogram ... | njRAT  | Firewall self-allow (IOA)          |
| 10 | Behavioral         | Geolocation lookup via cj.maxmind.com | Zeus   | Anti-sandbox check (IOA)           |
| 11 | Suricata SID       | 2028703 (ET POLICY DynDNS)            | njRAT  | Default rule confirms domain C2    |
| 12 | Suricata SID       | 2015474 (ET MALWARE ZeroAccess)       | Zeus   | Default rule confirms IP-based C2  |

### Why was interactive detonation more important here than trusting an automated sandbox run?

Automated sandboxes detonate a sample and produce a report without any operator involvement, which risks missing behaviour gated behind a UAC prompt, a dialog box, or a delay the malware author added specifically to outlast a short automated timer. Both sessions here were checked for genuine execution rather than assumed — Fake Net was enabled before detonation specifically so that C2 traffic could be observed even though the real command-and-control infrastructure behind decade-old samples is long offline, and the resulting network activity (DynDNS queries, P2P UDP traffic) would not have appeared at all had the sample only appeared to run without actually reaching its network-facing code paths.

### What is the practical difference between njRAT's and Zeus's command-and-control mechanisms, and why does that difference matter for detection?

njRAT resolves a domain name (\*.zapto.org) through a legitimate Dynamic DNS provider — the attacker can change which IP that domain points to at will, so a detection rule keyed only on IP address would go stale quickly, but a rule matching the domain pattern itself remains valid. Zeus/ZeroAccess instead uses a fixed IP address and sends encrypted binary data over the DNS port without it being real DNS — a

domain-based rule would never fire against this sample at all, so the correct detection approach is matching the destination IP, port, and protocol instead. The two samples therefore require structurally different Suricata rules, which is exactly why they were deliberately paired for this week's analysis.

#### **DELIVERABLES SUBMITTED**

Process tree screenshots (both samples) · Network Threats + threat detail expansions (both samples) · File-system activity screenshots (both samples) · Raw stream payload hex dump (Zeus) · MITRE-technique process detail panels (both samples) · Consolidated 12-row IOC/IOA table · PCAP + IOC exports + public share links (both samples)

# Day 34 — Detection Engineering (Suricata)

*Concepts: offline PCAP replay · custom signature authoring · isolated-rule verification*

## Objective

---

Convert the IOCs recovered in Days 32–33 into working Suricata detection logic, verified by replaying each sample's PCAP offline — the correct method for testing against cloud-sandbox traffic, since the malware executed on ANY.RUN's infrastructure rather than the local lab network.

## Environment Verification

---

Suricata 8.0.6 was confirmed installed, and the full Emerging Threats (ET) ruleset was downloaded via suricata-update — 52,725 rules total, confirmed loading cleanly:

```
suricata -T -c /etc/suricata/suricata.yaml -v
→ Info: detect: 2 rule files processed. 52725 rules successfully loaded, 0
failed, 0 skipped
→ Notice: suricata: Configuration provided was successfully loaded.
```

## njRAT PCAP Replay — Default Ruleset

---

```
suricata -r al29669d-...-cd1c8047dc34.pcap -l <logdir> -k none
→ i: pcap: read 1 file, 1109 packets, 275963 bytes
```

Result: the default ruleset fired SID 2028703 (ET POLICY DynDNS Query to \*.zapro.org) 8 times — independently confirming the ANY.RUN finding via offline replay. Two additional signatures fired repeatedly (SID 2200121 "Ethertype unknown" and SID 2210058 "STREAM suspected RST injection") and were identified as sandbox-network noise (virtual-switch LLDP/STP frames and a TCP anomaly from the virtual environment) rather than malicious behaviour, and are documented as such rather than miscounted as findings.

## njRAT Custom Suricata Rule — SID 9000004

---

```
alert dns any any -> any any (msg:"CUSTOM RAT C2 DNS Query to zapto.org DynDNS
domain"; dns.query; content:"zapto.org"; nocase; endswith; classtype:trojan-
activity; sid:9000004; rev:1;)
```

Basis: targets the exact DynDNS domain from IOC #3, using the dns.query buffer with endswith so any \*.zapro.org subdomain the malware rotates through will still match.

**Verification — combined ruleset replay:** fired 8 times, matching the exact timestamps and ports of the default ET rule — confirming both rules key off the same real traffic.

**Verification — isolated replay (-S local.rules only, no default ruleset loaded):** fired the same 8 times with nothing else in the log — proving the custom rule works independently, satisfying the task's isolation-test requirement.

```
suricata -r al29669d-...pcap -l <logdir> -k none -S
/etc/suricata/rules/local.rules
→ [1:9000004:1] CUSTOM RAT C2 DNS Query to zapto.org DynDNS domain {UDP}
192.168.100.12:61963 -> 192.168.100.2:53
(repeated 8 times, exact match to combined-ruleset run, no other alerts
present)
```

## Zeus Custom Suricata Rule — SID 9000005 (Drafted)

---

```
alert udp any any -> 85.114.128.127 53 (msg:"CUSTOM Zeus ZeroAccess C2 UDP Traffic"; content:"|37 CF EC 34|"; classtype:trojan-activity; sid:9000005; rev:1;)
```

Basis: targets the exact destination IP and port from IOC #4, with a content match on the first four bytes of the captured encrypted payload as an additional fidelity check beyond IP/port alone.

### STATUS — LIVE VERIFICATION PENDING

*This rule is written and structurally sound — same format as the verified njRAT rule, keyed to the confirmed Zeus IOC exactly. It could not be loaded and replay-tested against the Zeus PCAP because the analysis machine began experiencing recurring motherboard-level hardware failure (repeated VM crashes under load) partway through this session. The same three-step verification already proven for njRAT (config test, combined-ruleset replay, isolated -S replay) remains outstanding and will be completed once the hardware issue is resolved.*

## Host-Behaviour Gap — Why Suricata Cannot Detect Everything Observed

---

None of the following can be detected by Suricata, because Suricata inspects network traffic only and these are host-level events with no network signature: njRAT's Startup-folder persistence write, njRAT's firewall rule addition via netsh, or Zeus's DLL drop for search-order hijacking. Each of these requires host-based detection via Wazuh — file integrity monitoring on the Startup and Temp paths, and command-line/process monitoring for netsh.exe invocations. This is the exact handoff point to Day 34 Part 2, addressed in the following section.

### Why is PCAP replay in offline mode the correct approach here, rather than expecting a local Suricata capture to contain this traffic?

Both samples executed inside ANY.RUN's cloud sandbox infrastructure, not on the local lab network — a live Suricata capture on the Kali VM's own interfaces would never see this traffic at all, regardless of how long it ran. Suricata's -r flag reads packets from a saved file instead of capturing live, which is the standard, correct method detection engineers use when working from third-party sandbox reports rather than their own network taps.

### DELIVERABLES SUBMITTED

suricata --build-info and -T config verification · njRAT combined-ruleset replay output (fast.log) · njRAT custom rule source (SID 9000004) · njRAT isolated -S replay output confirming independent match · Zeus custom rule source (SID 9000005) — verification pending

## Day 34–35 — Wazuh Rules & Incident Response Plan (Status)

Concepts: host-based detection · MITRE-mapped custom rules · NIST SP 800-61 lifecycle

### REPORTING STANDARD FOR THIS SECTION

*Consistent with this internship's standard of documenting what was actually done rather than what was intended, this section states plainly that Day 34 Part 2 and Day 35 were not completed this reporting period, and explains why — rather than presenting fabricated Wazuh rule levels or an Incident Response Plan built on untested detection logic.*

### Day 34, Part 2 — Wazuh Custom Rules: Not Started

This requires writing at least 3 custom rules in `local_rules.xml` on the Wazuh Manager, covering at minimum two categories (persistence detection and process/command-line detection), each mapped to a MITRE ATT&CK technique, then testing each live against the Windows 10 agent by benignly recreating the trigger condition and confirming the alert fires with correct severity and MITRE mapping in the dashboard.

Draft rule logic — from evidence already gathered, not yet written into `local_rules.xml` or tested:

| # | Category               | Trigger (from evidence)                                                                        | MITRE Mapping |
|---|------------------------|------------------------------------------------------------------------------------------------|---------------|
| 1 | Persistence            | File creation in ...\\Startup\\ (matches njRAT's randomized-name drop)                         | T1547.001     |
| 2 | Process/command-line   | netsh.exe invoked with firewall add allowedprogram (matches njRAT)                             | T1562.004     |
| 3 | Persistence/masquerade | New .dll in AppData\\Local\\Temp matching a known system DLL name (matches Zeus's msimg32.dll) | T1574.001     |

This work cannot be completed or verified without the Windows 10 agent and Wazuh Manager, both of which run on the physical machine currently experiencing hardware failure.

### Day 35 — Incident Response Plan: Not Started

The IR Plan requires the completed Wazuh rules above as direct inputs to its Preparation and Post-Incident sections — writing it now would mean inventing rule levels and MITRE mappings that have not actually been built and tested, which defeats the purpose of an evidence-grounded IR document and would not hold up under review.

**What is already ready to go into the IR Plan once the Wazuh work above is complete:**

- Full attack timeline data — ANY.RUN timestamps for both sessions at millisecond granularity.
- The complete 12-row IOC/IOA table (Section 05 above).
- The confirmed detection-layer split: Suricata for network-layer C2 (DynDNS + P2P-over-DNS), Wazuh for host-layer persistence, firewall manipulation, and DLL hijacking.
- Eradication specifics already known precisely: the exact Startup-folder path, the exact Temp DLL name, and the exact firewall rule that needs reversing.

## Summary of Outstanding Work

---

| Item                        | Status                                               | Estimated Effort Once Unblocked |
|-----------------------------|------------------------------------------------------|---------------------------------|
| Zeus custom Suricata rule   | Written, needs live replay verification              | ~10 minutes                     |
| 3 Wazuh custom rules        | Drafted at concept level, not written or tested      | ~30–45 minutes                  |
| Full Incident Response Plan | Not started; all required evidence already collected | ~20–30 minutes                  |

### BLOCKING ISSUE

*The analysis machine is experiencing recurring motherboard-level crashes under VM load. This must be resolved — or the remaining work moved to a stable machine — before the items above can be completed and properly verified, rather than asserted without testing.*

# Technical SOC Summary

*Audience: Senior Security Engineers / SOC Director*

## 1. Sample Acquisition & Isolation

Two theZoo samples (njRAT v0.6.4, Zeus/ZeroAccess build) acquired via individual file download, hashed pre-analysis, and analysed only after hypervisor-level airgap confirmation (failed connectivity test, not a passing one). Sample permissions locked to 400 (read-only, non-executable).

## 2. Static Analysis Findings

file/exiftool/binwalk/strings run on both samples. Zeus's near-empty Unicode strings output (30 bytes) flagged as a packing indicator. Both samples cross-verified on VirusTotal: njRAT 62/70 (bladabindi/msil), Zeus 64/71 (zaccess/sirefef — note: dominant classification is ZeroAccess, not classic Zeus).

## 3. Dynamic Analysis Findings

Both samples detonated in ANY.RUN from inside the Kali VM (Fake Net enabled). njRAT: self-renaming process chain, Startup-folder persistence, netsh firewall bypass, C2 via \*.zapto.org (SID 2028703). Zeus: fake-installer masquerade (InstallFlashPlayer.exe), msimg32.dll drop for DLL search-order hijack, C2 via hardcoded IP 85.114.128.127:53/UDP disguised as DNS (SID 2015474), MaxMind geolocation anti-sandbox check.

## 4. Detection Engineering

Custom Suricata rule SID 9000004 (njRAT DynDNS) written, loaded, and verified via both combined-ruleset and isolated (-S) PCAP replay — 8/8 matches in both modes. Custom rule SID 9000005 (Zeus P2P C2) drafted but not yet live-tested due to hardware failure. Host-behaviour gap (Startup persistence, firewall bypass, DLL hijack) explicitly documented as requiring Wazuh, not Suricata.

## 5. Outstanding Work

Wazuh custom rules (3 minimum, 2 categories, MITRE-mapped) not yet written or tested; Incident Response Plan not yet written, pending the above. All supporting evidence for both is already collected and summarized in Section 07.

# Executive Summary

*Audience: Corporate Executives / Non-Technical Stakeholders*

**Overall progress: On track, with one delay to flag. This week the team analysed two real-world malicious programs in a fully isolated lab, extracted the specific fingerprints and behaviours attackers use, and built a working detection rule for one of them — proven to catch that exact threat.**

## Key Findings

- We safely examined two real malicious programs commonly used for remote spying and banking fraud, entirely inside a sealed-off practice environment with no risk to any real system.
- Both programs were confirmed malicious by independent industry sources, and we identified specific technical fingerprints — network addresses, file names, and behaviours — that can be used to detect them automatically in the future.
- We built and proved a working detection rule for one threat, confirming it correctly flags that exact malicious traffic pattern with no false triggers.
- A hardware failure on the analysis computer interrupted the final two days of planned work (a second detection rule and the incident-response planning document). The underlying research needed for both is already complete — only the write-up and testing remain, pending a working machine.

## Recommendation

Approve replacement or repair of the analysis machine as a priority, since it is now the sole blocker on this week's two remaining deliverables — both of which have a clear, short path to completion once hardware access is restored.

# Key Learnings & Reflection

Beyond the specific tooling exercised this week — theZoo acquisition, static and dynamic analysis, and Suricata rule authoring — several broader lessons emerged.

## **A near-empty result is itself a finding, not a dead end**

Zeus's Unicode strings pass returning only 30 bytes of punctuation could have been logged as "nothing found" and left there. Instead it was interpreted correctly as evidence the sample is packed — its real content only exists in memory at runtime — which directly justified the emphasis placed on capturing dynamic behaviour closely in the following two days. Absence of data is only a non-finding if it isn't interrogated for what it implies.

## **Isolated verification is what separates a working rule from a coincidental match**

The njRAT custom rule firing alongside the default ET ruleset was encouraging but not, by itself, proof the custom rule specifically worked — the default rule could have been doing all the work. Replaying with only the custom rule loaded via -S, and confirming the exact same 8 matches with nothing else present, is what actually proved the rule functions independently. This distinction — correlation versus isolated proof — is the same discipline that later made it easy to recognise the Zeus rule genuinely needed the same two-step verification before it could be called complete, rather than assumed correct because it followed the same template.

## **Two different C2 mechanisms in one week is a better lesson than two similar ones**

Pairing a domain-based DynDNS sample with a hardcoded-IP encrypted-P2P sample meant the two custom Suricata rules had to be structurally different from each other — one matching DNS query content, one matching destination IP/port and payload bytes. Had both samples used the same C2 pattern, the second rule would have been a copy-paste exercise rather than a genuine second data point on how detection logic has to adapt to what the traffic actually looks like.

## **Documenting a blocker honestly is more useful than working around it invisibly**

When the analysis machine began failing under load partway through Day 34, the choice was between quietly padding the report with untested, invented Wazuh rules and an IR Plan built on nothing, or stating plainly what was and wasn't done and why. The second option is the one that actually holds up under review — a reviewer can trust every verified claim in this report precisely because the unverified parts are labelled as such rather than blended in.

# Closing Statement

This concludes the Week 5 deliverable report for the Blue Team Internship, covering the acquisition and airgapped isolation of two real malware samples, full static and interactive dynamic analysis of both, and the conversion of recovered indicators into a verified custom Suricata detection rule. Day 34 Part 2 (Wazuh custom rules) and Day 35 (Incident Response Plan) were not completed this reporting period due to a hardware failure on the analysis machine partway through Day 34 — this is stated directly here and in Section 07 rather than papered over, consistent with this internship's reporting standard of describing what was done, what was observed, and what was concluded, including friction and incomplete work.

All screenshots, command outputs, and figures included in this report reflect commands actually executed and outputs actually observed in the lab environment during Days 29 through 34. No Wazuh dashboard output, rule-firing screenshot, or Incident Response content has been fabricated to simulate completion of Day 34 Part 2 or Day 35; both remain explicitly marked as outstanding pending hardware access.

| <b>SUBMITTED BY</b>     | Malaika Azhar                                       |
|-------------------------|-----------------------------------------------------|
| <b>TRACK</b>            | Blue Team                                           |
| <b>PROGRAM</b>          | Blue Team Internship — Phase One                    |
| <b>REPORTING PERIOD</b> | Week 5 · Days 29–35 (34–35 partial, see Section 07) |

## Appendix A — Deliverables & Screenshot Index

*Full reference list of evidence included in this report*

### Days 29–30 — Sample Acquisition & Environment Lockdown

---

| Figure    | Evidence                                                   |
|-----------|------------------------------------------------------------|
| 3.1       | Kali connectivity confirmed pre-download                   |
| 3.2       | 700-permission sample directory listing                    |
| 3.3 / 3.4 | theZoo repository browsing, both samples                   |
| 3.5       | Individual wget download (no git clone)                    |
| 3.6       | Extraction confirmation (njRAT ZIP contents)               |
| 3.7       | Hash table cross-checked against theZoo's published values |
| 3.8       | Failed-ping airgap confirmation (correct method)           |
| 3.9       | 400-permission sample listing                              |
| 3.10      | Baseline PCAP capture, Host-Only interface                 |

### Day 31 — Static Malware Analysis

---

| Figure    | Evidence                             |
|-----------|--------------------------------------|
| 4.1       | file output, both samples            |
| 4.2       | exiftool metadata, both samples      |
| 4.3       | binwalk embedded-payload detection   |
| 4.4       | ASCII vs. Unicode strings comparison |
| 4.5 / 4.6 | VirusTotal results, both samples     |

### Days 32–33 — Interactive Dynamic Analysis

---

| Figure    | Evidence                                                   |
|-----------|------------------------------------------------------------|
| 5.1 – 5.3 | njRAT process tree (full chain, incl. netsh/conhost)       |
| 5.4       | njRAT file-system activity (Startup persistence)           |
| 5.5 / 5.6 | njRAT Network Threats + DynDNS threat detail               |
| 5.7       | njRAT windows.exe process detail, danger 100/100           |
| 5.8       | Zeus process tree (InstallFlashPlayer chain)               |
| 5.9       | Zeus file-system activity (msimg32.dll, obfuscated folder) |

| Figure      | Evidence                                                   |
|-------------|------------------------------------------------------------|
| 5.10 / 5.11 | Zeus Network Threats + ZeroAccess UDP threat detail        |
| 5.12        | Zeus raw stream payload hex dump                           |
| 5.13        | Zeus InstallFlashPlayer.exe process detail, danger 100/100 |

## Day 34 — Detection Engineering

---

Suricata config verification, njRAT combined and isolated replay outputs, and both custom rule sources are included as command-output code blocks in Section 06 rather than screenshots, per the terminal-log format used throughout this internship's Suricata/Wazuh sections.

---

## Report Verification Statement

---

Every figure listed above corresponds to a screenshot or terminal output captured directly from the lab environment during execution of the referenced task, with no simulated or reconstructed content. Both ANY.RUN sessions were genuine live detonations, not synthetic reports. Where a Day 34–35 item was not completed — the Zeus Suricata rule's live verification, all of the Wazuh custom rules, and the full Incident Response Plan — this is stated explicitly as outstanding work rather than presented as done.

## END OF REPORT

*This report totals eleven sections plus appendix covering the scope of Week 5 (Days 29–35, with 34–35 partially complete) of the Blue Team Internship, Phase One, submitted for review.*